

Week 3: Object-Oriented Programming

Valid planning-domain objects, explicit state ownership, and collaboration

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

Week 2 represented a path as a numerical array and verified functions that measured and sampled it. That representation is effective for numerical calculation, but a larger program must also answer questions that an array cannot answer by itself: Which row is the start configuration? Which path belongs to which planning problem? Is a failure result allowed to contain a path? Who is allowed to append an execution result, and which object owns that changing history?

Object-oriented programming gives these roles names and places related data and operations behind explicit boundaries. The purpose is not to replace every function with a class. The supplied numerical function remains a pure function. Classes organize domain meaning, preserve valid state, and make collaboration between components visible and testable.

This chapter develops one continuous example. A sequence of numerical configurations becomes a `Path`. A `PlanningProblem` composes a start, goal, and bounds. A `PlanResult` represents either a valid path or an explained failure. A mutable `PlanningRun` records compatible results. The final section contains Assignment 3, Part A and the complete Week 3 repository and LMS submission procedure.

After completing this chapter, a student should be able to

- distinguish a class, instance, attribute, method, and the conventional `self` parameter in an executed Python example.
- use `@dataclass` while explaining which methods it generates and which validation it does not perform.
- state and enforce constructor invariants in `__post_init__`.
- distinguish frozen value objects from one explicitly mutable state owner and avoid shared mutable defaults.
- choose inheritance for an actual is-a compatibility relationship and composition for the has-a relationships in this domain model.
- test valid construction, invalid construction, successful updates, and rejected updates that must leave state unchanged.
- complete, test, package, and submit the Week 3 work that Week 4 will extend at the JSON boundary.

The package does not generate a path. It does not check collisions, control a robot, parse JSON, or run a physics simulation. Those limitations are part of the contract, not missing claims.

1 Numerical data need domain meaning

1.1 The same numbers can play different roles

Consider the supplied Week 2 path:

Code excerpt - supplied coordinates

```
import numpy as np

path_xy = np.array([
    [0.0, 0.0],
    [1.0, 1.4],
    [2.4, 1.1],
    [3.2, 2.0],
], dtype=np.float64)
```

This excerpt creates `path_xy`. It does not print a result. Its shape `(4, 2)` states that four rows each contain two coordinates. The array supports interpolation and length calculations, but the type alone does not identify the first row as the required start, the final row as the required goal, or the rectangle within which every waypoint must remain. A second array with the same shape might be sensor samples, obstacle points, or a completely unrelated table.

A **domain object** gives a concept in the problem domain a name, stores the data that define it, and exposes operations that belong to its responsibility. For Week 3, the domain is a small planning model. A **configuration** is one point in the state space used by the planning problem. The classroom example uses two coordinates so that it can be drawn as a point-robot path. The object contract permits other positive dimensions because a later robot configuration may contain several joint coordinates. A tool-center point does not, by itself, specify every joint of a robot arm. Here the two coordinates, measured in metres, describe a planar point robot. The lesson is the software representation, not the conversion from tool coordinates to joint angles.

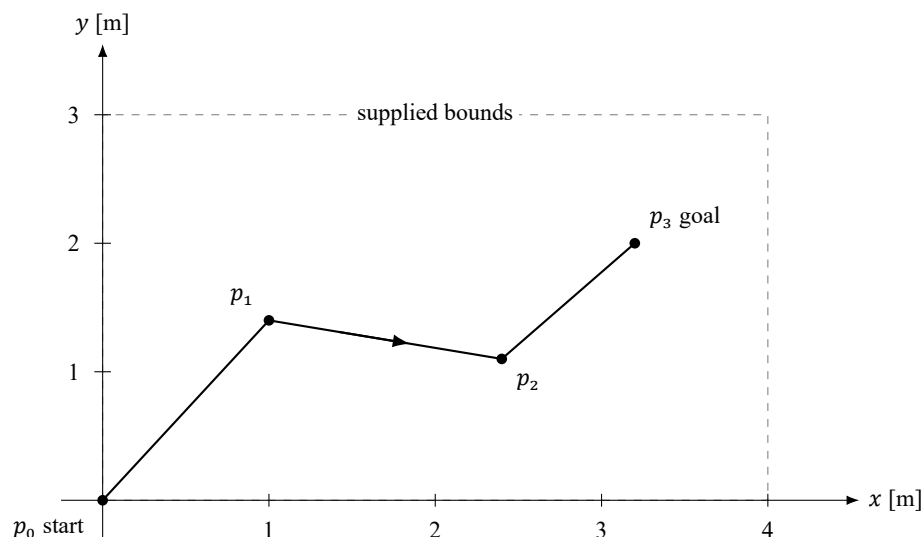


Figure 3.1: The supplied path is the ordered polyline p_0, p_1, p_2, p_3 . The dashed rectangle marks bounds $[0, 4] \times [0, 3]$ m. It is not an obstacle map.

The line in Figure 3.1 does not describe a planning algorithm. It displays supplied coordinates in order. Each point will become a **Configuration**, and their ordered collection will become a **Path**. Week 3 asks whether the program can construct a coherent set of objects around that supplied path and reject an incoherent set before it reaches later code.

1.2 Responsibility is narrower than “everything related”

An object should not absorb every operation that mentions its data. The `Path` object owns an ordered sequence of configurations and can expose its start, goal, array representation, and length. The numerical summation for the length is still performed by the supplied pure function `polyline_length`. This boundary gives two useful properties:

- domain code states what a path means and what must remain valid.
- numerical code performs one calculation without owning changing planning state.

Object-oriented programming and functional decomposition therefore cooperate. The design question is not “functions or classes?” but “which component should own this data, this rule, and this change?”

2 A class defines a kind of object

The first examples are small programs, not the unfinished Assignment package. Open **Anaconda Prompt**, activate the existing course environment, and make a practice folder outside any assignment repository. In the commands below, the folder is placed in your home directory:

Run - Anaconda Prompt, create the practice folder

```
conda activate applied-programming-2026
cd /d "%USERPROFILE%"
mkdir week03_practice
cd week03_practice
notepad 01_instances.py
```

If the folder already exists, skip `mkdir`. Save the program below as `01_instances.py`. Its execution command and output are paired below. Later examples specify a new filename when the class definition changes. They do not silently modify the class in an earlier file. On macOS or Linux, use the activated terminal, replace the home-directory command with `cd ~`, and use your text editor. The Python programs themselves are unchanged.

Activation selects the course Python and installed packages. The two `cd` commands change the working folder, `mkdir` creates the practice folder, and `notepad` opens the source file for editing. Running `python 01_instances.py` executes the saved file, not unsaved editor contents. Type shell commands in Anaconda Prompt and write Python code in the file. The program does not ask you to type coordinates while it runs. Its input data are the tuples already written in the source. Its `print` calls display the output shown below. These practice programs do not create submission files.

2.1 Class, instance, attribute, and method

A **function** is a piece of code that performs a task when called. An **argument** is an input value supplied by the calling code. A call supplies any arguments and runs the function body. The code making that call is the **caller**. Caller describes a role, not a different kind of Python object. It may be another function or code at the top level of a script. A **return** statement sends a value back to that calling code. It does not display the value. `print` displays text in the execution window.

A **class** defines a new object type. An **instance** is one object created from that class. An **attribute** is a named value reached through the object. A **method** is a function associated with the class and called through an object. The following class illustrates these relationships before adding the full Assignment input contract:

Input

The tuples `(0.0, 0.0)` and `(3.2, 2.0)` are written in the calling code. The program creates two instances from these values without asking for keyboard input.

Code - 01_instances.py

```
class Configuration:
    def __init__(self, values):
        self.values = tuple(values)

    def dimension(self):
        return len(self.values)

q_start = Configuration((0.0, 0.0))
q_goal = Configuration((3.2, 2.0))

print(q_start.values)
print(q_start.dimension())
print(q_start is q_goal)
```

Run - Anaconda Prompt, week03_practice

```
python 01_instances.py
```

Output

```
(0.0, 0.0)
2
False
```

`Configuration` is the class. `q_start` and `q_goal` are two instances. The expression `q_start.values` reads an attribute. The expression `q_start.dimension()` calls a method. The final result is `False` because the two names refer to different instance objects. In `q_start = Configuration((0.0, 0.0))`, the class call receives the tuple and creates an instance. Python passes the new instance as `self` and the tuple as `values` to `__init__`. The initializer stores `self.values`. After initialization succeeds, `q_start` refers to the instance. The initializer itself does not return the instance. Assigning this attribute does not change `q_goal`.

For the second printed line, `q_start.dimension()` returns the integer 2 to the surrounding code, and `print` displays it. The method contains no printing operation. Replacing that line with `count = q_start.dimension()` would save 2 in `count` without displaying a line. This distinction between a returned value and printed output will also apply to the larger package.

2.2 What `self` means during a method call

In the method definition, `self` is the conventional name for the instance on which the method operates. Python supplies that instance when the method is called through an object. These two calls are equivalent:

Code excerpt - two equivalent method calls

```
q_start.dimension()
Configuration.dimension(q_start)
```

The first form is normal application code. The second form exposes the call mechanism: `q_start` becomes the

first argument. The name `self` is not a keyword, but following the convention makes Python code readable to other programmers.

Both expressions return 2. In a saved script, a bare expression does not display its return value. To inspect either call, put it inside `print(...)` in the same `01_instances.py` file after `q_start` has been created.

Example 3.1 Two instances must not accidentally share changing state

An instance attribute belongs to one instance. A class attribute belongs to the class and is therefore visible through every instance unless an instance overrides it. A mutable class attribute can create an unintended shared list. Save this independent example as `shared_history.py` in the practice folder and run `python shared_history.py`:

Code - `shared_history.py`

```
class BadPlanningRun:
    results = []

first = BadPlanningRun()
second = BadPlanningRun()
first.results.append("failed")

print(second.results)
print(first.results is second.results)
```

Run - Anaconda Prompt, `week03_practice`

```
python shared_history.py
```

Output

```
['failed']
True
```

The `append` call receives the string `"failed"`, changes the existing list, and returns `None`. It does not print the new list. The following `print(second.results)` exposes the change. Although the update was made through `first`, `second` observes it because both lookups reach the same class-level list. A result history is not shared course configuration. It belongs to one planning run. The later `default_factory` example will create a distinct list for every instance.

2.3 Instantiation should establish a usable initial state

Calling a class object creates an instance. When the class defines `__init__`, Python calls it to initialize the new instance. A constructor is therefore an important public boundary: downstream code should not have to ask whether an object was only half initialized.

The explicit class above is still incomplete. It accepts `Configuration()`, `Configuration((True, False))`, and `Configuration(1.0, float("nan"))`. Each value is legal Python, yet none is a valid configuration under this package contract. A class organizes syntax. The program must still implement engineering validity rules.

3 Data classes reduce boilerplate, not responsibility

3.1 Generated methods come from annotated fields

An annotation such as `values: tuple[float, ...]` says that the intended field value is a tuple containing floating-point numbers. The ellipsis permits different tuple lengths. The `@dataclass` decorator inspects these annotated fields and can generate methods such as `__init__`, `__repr__`, and `__eq__`. Save this independent example as `dataclass_basics.py`. It replaces the handwritten initializer in the previous example rather than extending it. Save it in `week03_practice` and run `python dataclass_basics.py`:

Code - `dataclass_basics.py`

```
from dataclasses import dataclass

@dataclass
class Configuration:
    values: tuple[float, ...]

q1 = Configuration((0.0, 0.0))
q2 = Configuration((0.0, 0.0))

print(q1)
print(q1 == q2)
```

Run - Anaconda Prompt, `week03_practice`

```
python dataclass_basics.py
```

Output

```
Configuration(values=(0.0, 0.0))
True
```

Both constructor calls receive `(0.0, 0.0)`. They create separate instances. `print(q1)` uses the generated representation, and `q1 == q2` compares their field values to produce `True`. No handwritten initializer, representation or equality method is needed here. Field order follows the annotations in the class definition.

The annotation does not generally check runtime values. Add the next two lines to the end of that file, save, and run it again. The earlier two output lines remain, followed by the new line below. The call still constructs an instance unless the program adds validation:

Code - append to `dataclass_basics.py`

```
bad = Configuration(values="not numerical coordinates")
print(bad)
```

Run - Anaconda Prompt, week03_practice

```
python dataclass_basics.py
```

Output (additional line)

```
Configuration(values='not numerical coordinates')
```

A type hint communicates an intended interface to readers and tools. It is not a replacement for runtime validation at a public engineering boundary.

3.2 An invariant belongs to every valid state

An **invariant** is a condition that must hold for every successfully constructed public object and remain true after every successful public update. The Week 3 `Configuration` invariant is:

- values form one non-empty dimension.
- every element is numerical, real, non-Boolean, and finite.
- the instance owns a normalized tuple of Python `float` values.

These clauses answer different questions. `Shape` validates representation. Real, finite values validate numerical usability. Rejecting Boolean values prevents `True` and `False` from being interpreted as coordinates merely because Python's Boolean type is related to integers. Tuple normalization establishes ownership.

3.3 `__post_init__` checks generated initialization

The generated data-class initializer assigns the annotated fields and then calls `__post_init__` when it is present. A manually written `__init__` does not gain that automatic call. This next complete practice program adds just one validity rule: the tuple must not be empty. Save it as `02_dataclass_property.py` in `week03_practice`. The complete Assignment class will also need the numerical checks stated above.

Read this file as construction followed by observation. Its `main` function supplies the input list and prints each observation. `@property` lets `q.dimension` compute the coordinate count without parentheses. `frozen=True` rejects later field assignment. This example uses `object.__setattr__` only during initialization to store the normalized tuple. The final `try/except` blocks deliberately attempt invalid operations and print the caught errors, allowing the program to continue. The following sections examine these mechanisms in detail.

Input

The calling code starts with `[0.0, 0.0]`, then changes that list and attempts two operations that should be rejected. All values are supplied in the file, not typed while it runs.

Code - 02_dataclass_property.py, definition

```

from dataclasses import FrozenInstanceError, dataclass

@dataclass(frozen=True)
class Configuration:
    values: tuple

    def __post_init__(self):
        owned = tuple(self.values)
        if not owned:
            raise ValueError("values must not be empty")
        object.__setattr__(self, "values", owned)

    @property
    def dimension(self):
        return len(self.values)

```

The initializer first establishes the stored tuple. The calling code below then checks that later changes to the original list do not replace its coordinates. Append it to the same file, below the class definition.

Code - 02_dataclass_property.py, calling code

```

def main():
    source = [0.0, 0.0]
    q = Configuration(source)
    source[0] = 99.0

    print("caller list:", source)
    print("stored tuple:", q.values)
    print("q.dimension:", q.dimension)
    print("equal values:", q == Configuration((0.0, 0.0)))

    try:
        Configuration(())
    except ValueError as error:
        print("empty input rejected:", error)

    try:
        q.values = (9.0, 9.0)
    except FrozenInstanceError:
        print("field reassignment rejected: FrozenInstanceError")

if __name__ == "__main__":
    main()

```

Save the file and run it from week03_practice:

Run - Anaconda Prompt, week03_practice

```
python 02_dataclass_property.py
```

Output

```
caller list: [99.0, 0.0]
stored tuple: (0.0, 0.0)
q.dimension: 2
equal values: True
empty input rejected: values must not be empty
field reassignment rejected: FrozenInstanceError
```

The input list originally contains `[0.0, 0.0]`. After construction, changing its first element to `99.0` changes the first printed line but not the stored tuple. The two coordinates give dimension 2, and another object with the same coordinates compares equal. The empty input raises `ValueError` instead of producing a usable configuration. The matching handler prints the fifth line, then execution continues to the forbidden field assignment. These two rejection lines are ordinary printed messages from exception handlers, not unhandled tracebacks. They are expected observations, not defects to repair in this practice file.

3.4 A property reads like an attribute but runs a getter

Earlier, `q_start.dimension()` explicitly called a method. In this new definition, `@property` makes `q.dimension` call the getter without parentheses. Both compute a length. The public spelling has changed, so the two definitions must not be mixed. From this point onward the Assignment interface uses `q.dimension`, while `q.as_array()` and `path.length()` remain method calls.

No setter is supplied for `dimension`, so callers cannot assign a new dimension through this property. That does not make every object returned by a property immutable. A getter that returned a list would still expose that list. The next section addresses who owns such mutable data.

4 State ownership must be visible in the design**4.1 Frozen fields and owned values solve different problems**

The last practice definition added `frozen=True`. The package uses the same option for configuration, bounds, path, problem, and result objects. The last `try` block in the program attempts field assignment and catches `FrozenInstanceError`. The essential operation is:

Code excerpt - the rejected assignment

```
q = Configuration((0.0, 0.0))
q.values = (9.0, 9.0) # FrozenInstanceError
```

This behavior emulates read-only fields. It is not deep immutability. If a frozen field stores a mutable list or a caller-owned NumPy array, that nested object can still change. The following design is therefore unsafe even though the data class is frozen:

Code - array_aliasing.py

```

from dataclasses import dataclass
import numpy as np

@dataclass(frozen=True)
class UnsafeConfiguration:
    values: np.ndarray

source = np.array([0.0, 0.0])
q = UnsafeConfiguration(source)
source[0] = 99.0
print(q.values)

```

Run - Anaconda Prompt, week03_practice

```
python array_aliasing.py
```

Output

```
[99.  0.]
```

Save the complete array example above as `array_aliasing.py` and run `python array_aliasing.py`. The field was never reassigned. The array object stored in the field was mutated through another name. This is **aliasing**: multiple references lead to the same mutable object.

The practice class has shown the mechanism. The Assignment class must apply it to a stronger public contract. Its `ConfigurationLike` annotation names the allowed sequence or array input. Its internal `_normalized_values` helper collects the repeated representation and numerical checks. This helper is implemented in TODO 1, not supplied by Python. It must reject a Boolean before combining mixed values into an array, because `[True, 1.0]` can otherwise become two floating-point numbers. Conversion errors from a ragged sequence must also be reported as `PlanningModelError`, the package's public invalid-input exception.

The next block is an *Assignment implementation excerpt*, not a standalone practice program. The full module supplies its imports and other definitions. It validates the input and normalizes it to an owned tuple inside `__post_init__`. Because the data class is already frozen while post-initialization runs, it uses `object.__setattr__` for that one controlled normalization:

Code excerpt - Assignment Configuration

```

@dataclass(frozen=True)
class Configuration:
    values: ConfigurationLike

    def __post_init__(self) -> None:
        owned = _normalized_values(self.values, name="values")
        object.__setattr__(self, "values", owned)

```

This is not permission for ordinary callers to change the object. It is an implementation technique used while the constructor establishes the final valid representation.

4.2 Returning a copy protects the object's internal representation

Numerical callers still need a NumPy array. This next method belongs to the same `Assignment Configuration` class. It returns a new array each time:

Code excerpt - the same class, `as_array`

```
def as_array(self) -> NDArray[np.float64]:
    return np.array(self.values, dtype=np.float64, copy=True)
```

Read the caller below as a preview. Return to run it after the Assignment installation procedure and TODO 1–2 are complete. Save the caller as `array_copy.py` in `week03_practice`, then run `python array_copy.py` there. Without installation, the import fails with `ModuleNotFoundError`. An installed but unfinished package instead reaches `NotImplementedError` at a TODO.

Code - `array_copy.py`, after TODO 1–2

```
from ap_week03_planning import Configuration

q = Configuration((1.0, 2.0))
exposed = q.as_array()
exposed[0] = 100.0

print(exposed)
print(q.values)
```

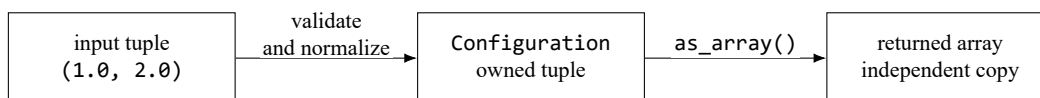
Run - Anaconda Prompt, `week03_practice`

```
python array_copy.py
```

Output

```
[100.    2.]
(1.0, 2.0)
```

The constructor receives `(1.0, 2.0)`. The method returns a separate array to `exposed`. Changing its first element to `100.0` changes the first output line, while the stored tuple on the second line remains `(1.0, 2.0)`. This is an ownership guarantee. It costs an allocation, so a performance-critical system would measure the trade-off. Week 3 prioritizes a small and unambiguous public contract.



Changing the returned array does not change the stored tuple.

Figure 3.2: The input tuple becomes the object's owned values. The caller receives a separate array from `as_array()` and can change it independently.

4.3 One mutable object owns result history

Not all state should be frozen. A planning run accumulates attempts in order. Before composing the full planning model, use a smaller history that stores text labels. Save the following complete program as `practice_history.py` in `week03_practice`, then run `python practice_history.py`. This class is a teaching example, not the Assignment implementation.

Input

The caller below supplies "first attempt" and a blank string. These are text labels used to expose ownership and rejection, not the richer `PlanResult` objects required in the Assignment.

Code - `practice_history.py`, definition

```
from dataclasses import dataclass, field

@dataclass
class PracticeHistory:
    _labels: list[str] = field(default_factory=list, init=False, repr=False)

    def record(self, label: str) -> None:
        if not isinstance(label, str) or not label.strip():
            raise ValueError("label must be a non-empty string")
        self._labels.append(label)

    @property
    def results(self) -> tuple[str, ...]:
        return tuple(self._labels)
```

Continue in the same file with this caller. The final two lines are the script entry-point convention used in Week 1. Running the file calls `main`, whereas importing its class for tests does not run the demonstration.

Code - `practice_history.py`, calling code

```
def main():
    first = PracticeHistory()
    second = PracticeHistory()
    earlier_snapshot = first.results

    first.record("first attempt")
    print("first history:", first.results)
    print("second history:", second.results)
    print("earlier snapshot:", earlier_snapshot)

    before = first.results
    try:
        first.record(" ")
    except ValueError as error:
        print("invalid update rejected:", error)

    print("history unchanged:", first.results == before)

if __name__ == "__main__":
    main()
```

After saving both parts in the same file, run it from the practice folder:

Run - Anaconda Prompt, week03_practice

```
python practice_history.py
```

The output makes the separate histories and rejected update visible:

Output

```
first history: ('first attempt',)
second history: ()
earlier snapshot: ()
invalid update rejected: label must be a non-empty string
history unchanged: True
```

`first.record("first attempt")` receives a label and returns `None` after appending it only to `first`'s internal list. Reading `first.results` returns a tuple snapshot. The later `print` displays that snapshot. The empty tuple on the second line shows that `second` owns another history. The third line is the snapshot taken before the append, so it also remains empty. The blank-string input is rejected, and the last `True` confirms that the history after rejection equals the history saved before the call. The stored history, the returned snapshot and its printed representation are related but distinct.

`default_factory=list` supplies the function `list` for the generated initializer to call for each new instance. It does not supply one already-created `list()` object to share. `init=False` keeps the internal list out of the public constructor arguments. `repr=False` omits it from the generated textual representation. Neither option makes the list private by itself.

The leading underscore says direct external use is not part of the public interface. The `results` property returns a tuple snapshot. Appending a later result creates a new history state, but does not change a previously returned tuple. This is not the shared-data view introduced for NumPy arrays in Week 2. Here the list structure is deliberately hidden.

4.4 Validate before mutation

In `PracticeHistory.record`, the type and non-empty checks precede `append`. If a check raises an exception, the `append` is never reached. The demo first records a valid label, remembers the history, then tries an invalid label. The unchanged history is the important result, not merely the presence of an error message. This is a **failure-atomic update**: at this public boundary the operation either succeeds or leaves state unchanged. It is not a claim about thread safety or an entire database transaction.

The Assignment applies the same ordering to richer objects. `PlanningRun` owns a `PlanningProblem` and its own initially empty list of `PlanResult` objects. Its `record_result` method must first check that the argument is a result for the same problem. Only then may it append. `results` returns a tuple snapshot, and `latest` returns the last result object, or `None` while history is empty. Copying the small practice class without adding these relationships would not complete TODO 8. The next section defines those relationships.

5 Inheritance and composition express different relationships

5.1 Inheritance is an is-a relationship

A derived class inherits accessible behavior from a base class and may refine it. Week 3 uses one narrow, practical relationship:

Code excerpt - public error type

```
class PlanningModelError(ValueError):
    """A public planning-domain value violated its contract."""
```

A `PlanningModelError` is a `ValueError`. Code that knows the course package can catch the specific type, while a broader caller can still catch `ValueError`.

Inheritance is not used to force unrelated domain nouns into a hierarchy. A path is not a specialized configuration, and a planning problem is not a specialized path. Making those classes inherit from one another would promise substitutability that the domain does not support.

5.2 Composition is a has-a relationship

Composition means that one object holds other objects as parts of its state. A `PlanningProblem` has a start configuration, a goal configuration, and configuration bounds. The following is its field declaration from the Assignment module. It is not a complete standalone class definition:

Code excerpt - `PlanningProblem` fields

```
@dataclass(frozen=True)
class PlanningProblem:
    problem_id: str
    start: Configuration
    goal: Configuration
    bounds: ConfigurationBounds
```

Its post-initialization validates relationships among the parts. The start, goal, and bounds must have the same dimension, and both endpoints must lie inside the bounds. Each component first validates its own local invariant. The composing object then validates the cross-object invariant.

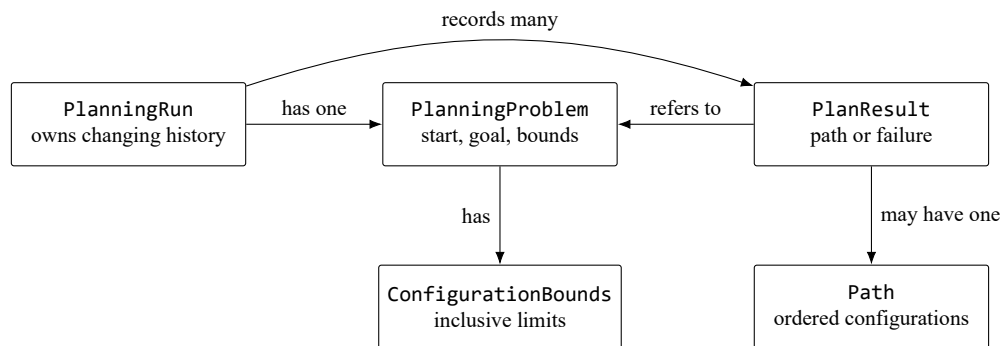


Figure 3.3: Each object validates its local state. Composing objects validate the relationships among those parts.

The diagram is a responsibility map, not a runtime sequence. At runtime a caller constructs objects in dependency order: configurations before bounds and paths, a problem before a result, and a compatible result before recording it in a run.

For example, a lower bound of $(0.0, 0.0)$ and an upper bound of $(0.0, 3.0)$ are individually valid configurations but an invalid pair: the first interval has zero width. `ConfigurationBounds` must check the parts' types, equal dimensions, and strict lower–upper ordering during initialization. Its `contains` method then uses inclusive limits, so a point exactly on an otherwise valid rectangle edge is inside. The strict rule creates a usable region. The

inclusive rule decides membership in that region. For bounds $(0.0, 0.0)$ to $(4.0, 3.0)$, `contains` returns `True` for `Configuration((4.0, 3.0))` and `False` for `Configuration((5.0, 1.0))`. An argument of the wrong type, such as a bare tuple, or the wrong dimension raises `PlanningModelError` instead.

5.3 A result uses cross-field and cross-object invariants

A `PlanResult` contains a problem and either a path or no path. This Assignment excerpt shows its fields and derived property. Validation belongs to the same class's `__post_init__` and is omitted from this excerpt:

Code excerpt - `PlanResult` fields and property

```
@dataclass(frozen=True)
class PlanResult:
    problem: PlanningProblem
    path: Path | None
    message: str = ""

    @property
    def succeeded(self) -> bool:
        return self.path is not None
```

The representation avoids a separate Boolean that could contradict the path. Success is derived from the presence of a path. The constructor then enforces the following rules:

Result form	Required relation	Rejected contradiction
Successful	A path exists. Its dimension and endpoints match. Every waypoint is inside bounds.	path begins elsewhere, ends elsewhere, uses another dimension, or leaves the bounds
Failed	The path is <code>None</code> . A non-empty explanation exists.	no path and no failure explanation
Both	message is a string	numerical or structured message silently accepted as prose

A failed result is still a valid `PlanResult` object: it stores `path=None`, a non-empty explanation, and has `succeeded=False`. A compatible run may record it. In contrast, a constructor exception means no usable result was created because its fields violated the contract. An explained failure and an invalid object are not the same outcome.

This course model does not claim that a path inside rectangular bounds is collision-free. Bounds are the only validity information supplied in Week 3. A later collision checker must add a different contract.

5.4 Collaboration delegates rather than duplicates

The complete caller later in this chapter constructs a supplied path, attaches it to a result, and records that result in a run. Its work passes through several responsibilities:

1. each `Configuration` owns finite coordinates.
2. `Path` owns order and dimensional consistency.
3. `PlanningProblem` owns endpoint and bounds relationships.
4. `PlanResult` connects a path to its problem.
5. `PlanningRun` owns the changing result sequence.

6. `Path.length()` delegates the calculation to the supplied pure function.

No one object is described as a manager of everything. Narrow responsibilities make failures easier to locate and tests easier to state.

6 Tests specify construction and update behavior

6.1 Object tests ask more than whether a method returns a number

Week 2 tests focused on numerical input, output shape, tolerance, and no-mutation behavior. Week 3 retains those ideas and adds object-state questions:

Test category	Question	What the test establishes
Valid construction	Can a coherent object be created?	normalized tuple, dimension, generated representation
Invalid construction	Is an impossible object rejected immediately?	empty configuration, reversed bounds, mismatched endpoints
Ownership	Can a caller mutate internal state through an alias?	source array mutation and returned-array mutation
Valid update	Does one compatible result appear in order?	result count and <code>latest</code> identity
Invalid update	Is failure atomic?	exception plus unchanged history
Instance isolation	Do separate objects own separate mutable defaults?	update one run while the other remains empty
Collaboration	Do composed objects preserve their relationships?	result path matches its problem

6.2 Test the public effect, including the exception type

The small history from Section 4 is already executable, so it is a useful place to verify these ideas before starting the Assignment. Save the following file as `test_practice_history.py` beside `practice_history.py`. The import reads the class from that local file. Each test creates its own objects, so no hidden setup or previously executed cell is required.

Input

Each test constructs its own history and supplies the labels or invalid value shown in its body. No data file or keyboard input is needed.

Code - test_practice_history.py, first two tests

```
import pytest

from practice_history import PracticeHistory

def test_normal_record_keeps_order_and_returns_a_snapshot():
    history = PracticeHistory()
    history.record("first attempt")
    snapshot = history.results
    history.record("second attempt")

    assert snapshot == ("first attempt",)
    assert history.results == ("first attempt", "second attempt")
    assert isinstance(history.results, tuple)

def test_invalid_non_string_label_is_rejected():
    history = PracticeHistory()

    with pytest.raises(ValueError):
        history.record(123)
```

The same file continues with two state-related tests. Neither test reuses an instance from the first two tests.

Code - test_practice_history.py, remaining tests

```
def test_two_instances_have_independent_histories():
    first = PracticeHistory()
    second = PracticeHistory()
    first.record("first attempt")

    assert first.results == ("first attempt",)
    assert second.results == ()

def test_rejected_update_leaves_existing_history_unchanged():
    history = PracticeHistory()
    history.record("valid attempt")
    before = history.results

    with pytest.raises(ValueError):
        history.record(" ")

    assert history.results == before
```

Run the tests from week03_practice. In `python -m pytest`, `-m` runs the named module using the selected Python interpreter. The `-q` option requests a shorter test report. The filename limits this run to the four tests saved beside the practice class:

Run - Anaconda Prompt, week03_practice

```
python -m pytest -q test_practice_history.py
```

Output (summary)

```
4 passed
```

The first test observes both order and an earlier snapshot. The second uses `pytest.raises(ValueError)` because rejecting a number is part of the practice contract. The third establishes separate ownership between instances. The fourth checks an exception *and* unchanged state. Simply seeing an exception would not catch an implementation that appended the invalid value before rejecting it.

These four practice tests are not the eight independent tests required for Assignment 3, Part A. The Assignment tests must call the actual planning package and use independent cases. Its public failure type is `PlanningModelError`, so those rejection tests use `pytest.raises(PlanningModelError)`. That check accepts the named exception type and its subclasses. It does not require an identical error message or expose the internal helper.

The supplied suite also uses parameterization to run the same check with several inputs. An empty coordinate sequence, a Boolean coordinate, and a non-finite coordinate have different reasons for rejection even when they share a test body. Input objects are not automatically copied by parameterization, which is another reason to keep mutations explicit.

6.3 A failure trace identifies the first violated contract

When a test fails, begin with the first failed node ID, then read the exception or assertion, the actual value, and the first line in the editable source. Do not begin by changing several classes at once. The starter first fails in `test_value_objects_reject_field_reassignment` during `Configuration` construction, before its field-reassignment assertion. The selected traceback lines identify the unfinished source operation:

Output excerpt - first failure

```
E   NotImplementedError: TODO 1: validate Configuration
src/ap_week03_planning/domain.py
```

The leading E marks pytest's error explanation, not a command to type. `NotImplementedError` means the intentional TODO is still reached. After an implementation is written, an `AssertionError` instead means a test's asserted relationship was false. For example, a history that changes after a rejected update would fail the fourth practice test's final assertion. Do not interpret both failures as the same diagnosis.

The first dependency is still incomplete. Bounds, paths, problems, and results all depend on valid configurations, so repairing a later class first would not produce a useful checkpoint.

6.4 Worked example: construct, record, and inspect one supplied path

The repository demo uses the same four two-dimensional points introduced in Week 2. It constructs objects. It does not search for the path. The complete caller below can be saved as `worked_objects.py` in the practice folder and run *after* the installed Assignment package has TODO 1–8 completed. Before completion, its first unfinished constructor will raise `NotImplementedError`. It is not another file to submit.

Input

All inputs are supplied in the code. The string `"demo-tool-path"` identifies the problem. The bounds are `(0.0, 0.0)` and `(4.0, 3.0)`. The four ordered coordinates, in metres, are the start, two interior points and the goal. No coordinate entry or input file is needed.

Code - worked_objects.py, construct the problem and path

```
from ap_week03_planning import (
    Configuration, ConfigurationBounds, Path,
    PlanningProblem, PlanResult, PlanningRun,
)

bounds = ConfigurationBounds(
    Configuration((0.0, 0.0)),
    Configuration((4.0, 3.0)),
)
problem = PlanningProblem(
    "demo-tool-path",
    Configuration((0.0, 0.0)),
    Configuration((3.2, 2.0)),
    bounds,
)
path = Path((
    problem.start,
    Configuration((1.0, 1.4)),
    Configuration((2.4, 1.1)),
    problem.goal,
))
```

Append the following lines to the same file. The supplied path becomes a `PlanResult`. The mutable run records that result, after checking that it belongs to this problem. The three `print` calls then read the recorded object rather than a separate list of display values.

Code - worked_objects.py, record and inspect the result

```
run = PlanningRun(problem)
run.record_result(
    PlanResult(problem, path, "validated supplied path")
)
print(f"path length: {run.latest.path.length():.6f}")
print(f"result count: {len(run.results)}")
print(f"succeeded: {run.latest.succeeded}")
```

Run - Anaconda Prompt, week03_practice

```
python worked_objects.py
```

Output

```
path length: 4.356407
result count: 1
succeeded: True
```

These are the three lines produced by the complete caller above. For the fuller supplied demonstration, return to the private repository root and run the protected module. It uses the same inputs but prints three additional descriptive lines:

Run - Anaconda Prompt, repository root

```
python -m ap_week03_planning.demo
```

Output

```
problem_id: demo-tool-path
configuration dimension: 2
waypoints: 4
path length: 4.356407
result count: 1
succeeded: True
```

The calculated length is approximately 4.35640661761539 m. The format `:.6f` displays six decimal places, giving 4.356407. The other lines expose the problem label, dimension, waypoint count, one accepted update and its success property. The demo prints these lines but writes no report files. None of the lines proves that an algorithm discovered the path or that a robot can execute it safely.

6.5 From printed values to inspectable files

The supplied output-generation script writes a JSON report and a browser-readable SVG. The SVG draws the four stored two-dimensional configurations and the rectangular bounds. It helps a human notice a swapped coordinate or missing waypoint, but visual plausibility is not the correctness criterion. Tests establish the public contract. The display makes one executed case inspectable.

Input

The generator uses the same supplied problem, bounds and four coordinates as the demo. It does not read a new input JSON file. Run it only after the package implementation is complete.

Run - Anaconda Prompt, repository root

```
python scripts/generate_outputs.py
```

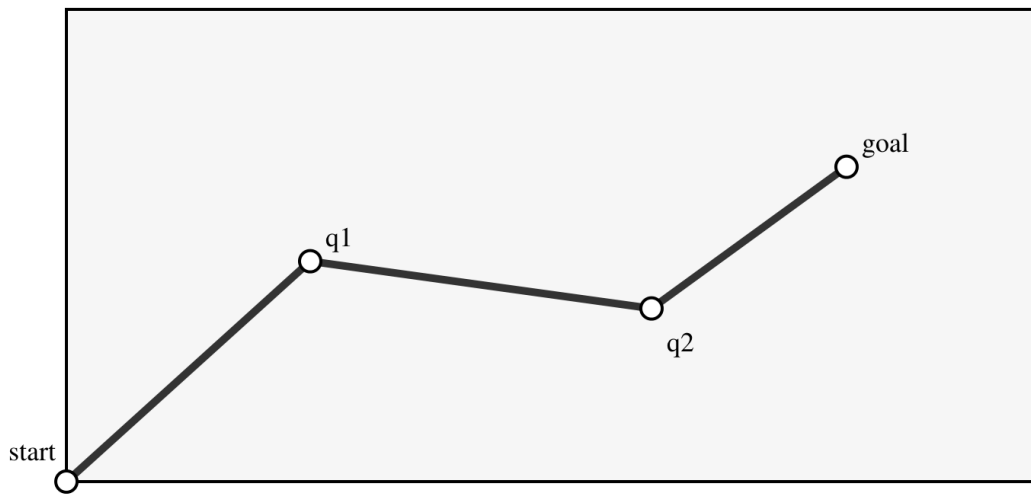
Output

```
[PASS] wrote artifacts\planning_objects_report.json
[PASS] wrote artifacts\planning_objects_preview.svg
[INFO] path_length=4.356407; result_count=1
```

Generated files

- `artifacts/planning_objects_report.json` stores the object values as structured text, including the full computed length.
- `artifacts/planning_objects_preview.svg` displays the same ordered points and bounds in a browser.

Problem: demo-tool-path



Display of supplied path objects; not a planner or physics simulation.

Figure 3.4: The generated SVG rendered from the completed instructor reference. The display represents stored points and bounds. It is not a planner result or a physics-simulation result.

The printed paths identify saved files, not commands to copy into the prompt. In the Assignment’s output-generation step, you will open the SVG, read the JSON and explain how both relate to these output lines. Rerunning the generator replaces these two files from the current object implementation.

Exercise 3.1 Classify a responsibility before writing code

For each behavior, state whether it belongs to a frozen value object, the mutable `PlanningRun`, or the supplied pure geometry function. Justify the owner in one sentence.

1. reject a configuration that contains NaN.
2. add a compatible result to chronological history.
3. sum Euclidean distances between consecutive rows.
4. reject a successful path whose goal differs from the problem goal.
5. give a caller a NumPy representation that it may safely mutate.

Exercise 3.2 Predict the aliasing outcome

Suppose a frozen data class stores a list directly. A caller appends to the original list after construction. Predict the stored field, then rewrite the constructor boundary so the object owns an immutable representation. Explain why `frozen=True` alone did not solve the first version.

Week 4 will begin from the object boundary established here: external scene data must be parsed and validated before it becomes a `PlanningProblem`. The JSON representation may change how data enters the package. It must not bypass the constructor invariants that now protect the domain objects.

7 Assignment 3, Part A: Planning Domain Objects

Complete the objects that hold a supplied path, reject invalid data, and keep each planning run's results separate. You are not writing a planner. The numerical length function, demo, drawing code, and execution tools are provided. Your work makes those callers operate on valid objects instead of unfinished TODOs. Section 7.6 gives the unchanged Week 3 submission requirements.

7.1 Starting state and required result

The starter has TODO 1–8 in two source files. Their `NotImplementedError` exceptions deliberately prevent unfinished code from appearing to work. Complete those bodies and write your own tests. The supplied demo will then construct the four-point path from Section 6.4, record one result, and report a path length of 4.356407 and `succeeded: True`. No new path is searched for.

The completed submission must contain:

- completed `domain.py` and `history.py` with all 76 published tests passing.
- at least eight independent student test functions and at least 84 passing tests in total.
- the generated JSON report and SVG display.
- the engineering note with all sections 0–6 completed.
- one built Week 3 wheel.
- a clean, pushed private-repository commit with active SSUMechE access or a correct on-time invitation awaiting acceptance.
- the three LMS fields identifying that Week 3 commit.

7.2 Files you edit and files you preserve

Edit only these four files. All source paths below are relative to your Week 3 repository root, the folder containing `README.md`.

Student-edited file	What you write
<code>src/ap_week03_planning/domain.py</code>	TODO 1–7, at the functions and properties named in Section 7.4
<code>src/ap_week03_planning/history.py</code>	TODO 8, the four <code>PlanningRun</code> bodies
<code>tests/test_student_evidence.py</code>	replace the placeholder with at least eight independent tests
<code>artifacts/engineering_note.md</code>	fill the existing sections using your results and explanations

Keep signatures, decorators, fields, public names, and package structure. Do not edit the published tests, `geometry.py`, exports, demo, visualizer, scripts, environment files, build metadata, Reading, README, or `PROTECTED_FILES.md`. The supplied tools generate the JSON, SVG and wheel. Commit those outputs without hand-editing them. The grader may restore protected files before checking your work.

7.3 Create the private Week 3 repository

If you already started, keep your existing private repository, local folder, and completed code. This revision clarifies the same TODOs and requirements. Do not clone again, overwrite your answers, or send another invitation.

For a new start, sign in to GitHub and open the course template:

Open in browser

```
https://github.com/SSUMechE/applied-programming-2026-week03-starter
```

Select **Use this template** then **Create a new repository**. Use your own account as Owner, select **Private**, and name it `applied-programming-w03-YOUR-STUDENT-ID`. Under **Settings** → **Collaborators** → **Add people**, invite exactly SSUMechE. Check whether the invitation is pending or access is active. You may work while acceptance is pending. Your own clone authentication and the instructor's access are separate.

Open **Anaconda Prompt**, not Python's `>>>` prompt. Replace the folder, username and student-ID placeholders below. The `set` commands give the URL short names for this command window.

Run - Anaconda Prompt, course parent folder

```
cd /d "C:\your-course-folder"
set "OWNER=YOUR-GITHUB-ID"
set "REPO=applied-programming-w03-YOUR-STUDENT-ID"
git clone https://github.com/%OWNER%/%REPO%.git
cd applied-programming-w03-YOUR-STUDENT-ID
git status
git remote -v
```

Both `origin` addresses must name your private Week 3 repository, not the public template. On macOS/Linux, use the Week 2 terminal commands and the HTTPS URL from your private repository's **Code** menu instead of the Windows `set` and `cd /d` syntax.

At the first HTTPS clone, sign in with the repository owner's account through Git Credential Manager. If credentials are requested, follow the Week 1–2 token guidance, not an ordinary GitHub password. Never include a token in a URL, file, screenshot or LMS entry. For `Repository not found`, check the URL and signed-in account.

Procedure 3.1 Install and verify the declared environment

From now on, run commands in the repository root. Reuse the working Week 1 course environment. Activation selects Python, the first `pip` command installs external packages, and `-e .` links this repository's package to the source you will edit. Saved TODO changes take effect on the next Python run.

Run - Anaconda Prompt, repository root

```
conda activate applied-programming-2026
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
```

Output (summary)

```
[PASS] Week 3 object-programming environment is ready.
```

`--no-build-isolation` uses the installed build tools, so install the requirements first. The verifier checks Python, required packages, the import location and public names. It does not execute unfinished TODO bodies or prove that their calculations are correct. Stop and resolve an installation error before attempting the baseline.

Only if the course environment does not exist, create it using the current template's corrected `environment.yml`, then repeat installation above:

Run - only when the named environment is missing

```
conda env create -f environment.yml
conda activate applied-programming-2026
```

Keep a working environment. If conda is not found, open Anaconda Prompt. For a DLL or application-control error, use the repair instructions in the current README. Keep your repository and security settings. If your repaired environment is named `applied-programming-w03`, activate that name in later windows.

The supported alternative is an installed Python 3.12, not another arbitrary Python version. In Windows Command Prompt, create `.venv` only if absent, activate it, then run the same pip commands and verifier:

Run - Python 3.12 alternative, repository root

```
py -3.12 -m venv .venv
.venv\Scripts\activate
```

Procedure 3.2 Record the intentional baseline before editing

Run - untouched starter, repository root

```
python scripts/run_baseline.py
```

Output (summary)

```
68 failed, 8 passed
```

The wrapper prints PASS when it recognizes this deliberately failing starting state. It does not report that the tests passed. Open `artifacts/engineering_note.md`, enter the summary and the first FAILED pytest node ID in section 0, and save with **Ctrl+S**. If you already edited TODOs, keep your recorded baseline. Do not undo your work to recreate it. A collection error or a different untouched-starter count needs diagnosis, not changes to protected tests.

```
self = Configuration(values=(1.0, 2.0))

    def __post_init__(self) -> None:
        # TODO 1: normalize values inside this frozen dataclass.
>       raise NotImplementedError("TODO 1: validate Configuration")
E       NotImplementedError: TODO 1: validate Configuration

src\ap_week03_planning\domain.py:35: NotImplementedError
```

Figure 3.5: Actual untouched-starter failure from the VS Code Debug Console. `domain.py:35` raises `NotImplementedError` before the test can check the object. This is missing implementation, not an incorrect numerical answer. The instructor's `-x` run stopped at 1 failed, 7 passed, unlike the full baseline above.

7.4 Complete TODO 1–8 in dependency order

Use your editor to open the file in Section 7.2 and search for the TODO number. Replace the corresponding `raise NotImplementedError(...)` body with your implementation. Keep the `def` line, decorators and declared fields. One TODO can occur in several bodies: complete every location named below. Save the file before testing. Do not paste the chapter's smaller demonstration classes over the supplied classes.

All rejected public inputs raise `PlanningModelError`, including failed array conversions. Constructors validate and store fields. Properties return values when read without parentheses. Ordinary methods such as `as_array()` are called with parentheses. None of these implementations needs to print a result. Use the existing checks below to observe their behavior. The paired checkpoints let later objects remain unfinished.

Procedure 3.3 Complete configurations and bounds - TODO 1–4

Work in `src/ap_week03_planning/domain.py`.

TODO 1: `_normalized_values` and `Configuration.__post_init__`. First make the helper return an owned tuple of Python floats. Accept non-empty one-dimensional real finite coordinates. Reject strings, Boolean values, complex values, object-dtype arrays, empty or wrong-shaped input, and `nan/inf`. Check Boolean elements before NumPy can turn a mixed input such as `[True, 2.0]` into numbers. Convert array-conversion errors to the public exception. Then call the helper from `__post_init__` and store its result in `values`, using the frozen-initialization pattern in Sections 3.3 and 4.1. For example, `Configuration([1, 2]).values` is `(1.0, 2.0)`. Later changes to the input list or array must not change it.

TODO 2: `Configuration.dimension` and `as_array`. Return the stored coordinate count from the property. Return a new `float64` array from the method, as in Section 4.2. For the preceding object, `dimension` is 2 and the array has shape `(2,)`. Changing that returned array must leave `values` unchanged.

Run after TODO 1–2 - repository root

```
python -m pytest -q tests/test_published_contract.py -k test_configuration_ -x
```

Output (summary)

```
21 passed, 55 deselected
```

Here `-k` selects matching test names. `-x` stops at the first failure. `deselected` means excluded from this run, not passed. Read the first failed test's input and assertion to locate the missing behavior. `E` marks failure-detail lines. Do not run the whole suite yet and mistake later `NotImplementedError` exceptions for a failure of these two TODOs.

TODO 3: `ConfigurationBounds.__post_init__` and `dimension`. Require lower and upper to be `Configuration` objects with equal dimensions. Require a strictly smaller lower value in every coordinate, then return that common dimension. For example, limits `(0, 0)` and `(4, 3)` form valid two-dimensional bounds. Limits `(0, 0)` and `(0, 3)` do not.

TODO 4: `ConfigurationBounds.contains`. Require a configuration of the matching dimension, then return a Boolean. Every coordinate must lie between its lower and upper limits, including the limits themselves. With the bounds above, `(4, 3)` is inside and `(4.1, 3)` is outside. An incompatible type or dimension raises the public exception instead of returning `False`. Reuse the validated objects rather than writing another coordinate-normalization routine.

Run after TODO 3–4 - repository root

```
python -m pytest -q tests/test_published_contract.py -k test_bounds_ -x
```

Output (summary)

```
11 passed, 65 deselected
```

Procedure 3.4 Complete path and problem composition - TODO 5–6

Continue in `domain.py`. Section 5 explains the object relationships.

TODO 5: six bodies in Path.

1. `__post_init__`: store the waypoint sequence as an owned tuple. Require at least two configuration objects with equal dimensions. Reject strings and invalid sequences. Changing the caller's original list must not change the stored waypoints.
2. `dimension, start, goal`: return the common dimension, first configuration and last configuration respectively.
3. `as_array`: return a new `float64` array with one waypoint per row, shape `(N, D)`. The caller may change it without changing the path.
4. `length`: pass that array to the supplied `polyline_length` and return its float result. Do not reimplement `geometry.py`.

For configurations `(0, 0)`, `(3, 0)`, `(3, 4)`, the path has dimension 2, array shape `(3, 2)`, and length `7.0`. The input is a sequence of configuration objects, not an unwrapped numeric array.

TODO 6: PlanningProblem. Implement `__post_init__`. Require a string `problem_id`, remove outer whitespace, and reject an empty result. Store the stripped ID. Require configuration objects for `start/goal` and a `ConfigurationBounds` object for `bounds`. Their dimensions must agree and both endpoints must be inside the bounds. Use the existing `contains` method to check membership.

Run after TODO 5–6 - repository root

```
python -m pytest -q tests/test_published_contract.py -k "test_path_ or test_problem_" -x
```

Output (summary)

```
17 passed, 59 deselected
```

Run this pair after both TODOs. Some path tests create a planning problem as test setup and therefore need TODO 6 even when checking a `Path` method.

Procedure 3.5 Complete results and independent history - TODO 7–8

TODO 7: PlanResult.__post_init__ and succeeded, in domain.py. Require a `PlanningProblem` and a string message. Store the message without outer whitespace. If `path` is `None`, require a non-empty explanation. Otherwise require a `Path` with the problem's dimension, matching start/goal and every waypoint inside the bounds. An empty message is allowed for success. The property returns `True` exactly when a path is present. These checks do not establish collision freedom.

TODO 8: four bodies in PlanningRun, in src/ap_week03_planning/history.py. Sections 4.3–4.4 show why the history belongs to one instance.

1. `__post_init__`: require a `PlanningProblem`. The supplied field(`default_factory=list, ...`) already creates a separate empty `_results` list for each run. Keep that field declaration.
2. `record_result`: require a `PlanResult` whose problem equals this run's problem. Validate before appending to `_results`. Reject an invalid result without changing the list. This method returns `None`, not the appended result.
3. `results`: return a tuple of the current history, not the mutable list. This tuple is a snapshot of the entries at that moment. Later appends must not add entries to an earlier returned tuple.
4. `latest`: return `None` when empty, otherwise the actual most recently recorded result object, not a newly constructed copy.

Run after TODO 7–8 - repository root

```
python -m pytest -q tests/test_published_contract.py -k "result or run" -x
```

Output (summary)

```
16 passed, 60 deselected
```

This filter includes result/history checks and matching dataclass metadata cases. It is not the full suite. Now run all published cases:

Run - all TODOs saved, repository root

```
python -m pytest -q tests/test_published_contract.py
```

Output (summary)

```
76 passed
```

This completes the published-code checkpoint. Independent tests, written explanation, generated files and submission still follow.

7.5 Add eight independent tests

In `tests/test_student_evidence.py`, replace `test_student_placeholder` with at least eight top-level `test_...` functions. Each test creates its input, calls your public object or method, and asserts the expected value, exception or unchanged state. Use new numerical values. Do not copy or merely rename published tests. Include all eight categories:

1. valid configuration normalization.
2. invalid representation or a non-finite coordinate.
3. inclusive bounds.
4. input aliasing or independence of a returned array.
5. path ownership or length.
6. an invalid composed problem or result.
7. two runs whose histories remain independent.
8. an invalid update that raises and leaves history unchanged.

Run - repository root

```
python -m pytest -q
```

Output (summary)

```
84 passed
```

The displayed count assumes exactly eight student cases. At least 84 must pass, with at least eight distinct student test functions. Skipped tests do not count. Each test must make a distinct claim supported by its assertions.

Procedure 3.6 Generate and interpret the report and display

The supplied demo uses the fixed path from Section 6.4. Run it after completing the TODOs. It prints six summary lines but writes no files.

Run - repository root

```
python -m ap_week03_planning.demo
```

Output - actual instructor-reference capture

```
problem_id: demo-tool-path
configuration dimension: 2
waypoints: 4
path length: 4.356407
result count: 1
succeeded: True
```

The capture is from the VS Code Debug Console. The same module prints these lines in Anaconda Prompt. To save the report and display from the same inputs, run the generator. No additional input file is needed.

Run - repository root

```
python scripts/generate_outputs.py
```

Output

```
[PASS] wrote artifacts\planning_objects_report.json
[PASS] wrote artifacts\planning_objects_preview.svg
[INFO] path_length=4.356407; result_count=1
```

Generated files

- artifacts/planning_objects_report.json: saved object values.
- artifacts/planning_objects_preview.svg: browser-readable display.

Run - open the display and read the saved report

```
start "" artifacts\planning_objects_preview.svg
type artifacts\planning_objects_report.json
```

If the SVG opens in an editor, open it from your browser instead. Check four ordered points, start/goal and rectangular bounds. In the JSON, check `problem_id`, dimension, endpoints, waypoint count, length, result count and success. The JSON length retains more digits than the six-decimal terminal display. These files report object data, not a planned, collision-free or physically verified motion. Rerunning the generator replaces only the two generated files, not your note or TODO code.

Procedure 3.7 Complete the engineering note

Open the existing note in your editor, or use this command:

Run - repository root

```
notepad artifacts\engineering_note.md
```

Fill the identity fields and replace every `REPLACE_ME`. Follow its existing sections rather than creating another document:

- **0:** the recorded baseline and first failed pytest node ID.
- **1:** frozen value-object responsibilities and why only `PlanningRun` owns mutable history, in two or three sentences.
- **2:** one constructor invariant and one state-update invariant, each linked to your own independent test name.
- **3:** the error inheritance relationship and one composition relationship.
- **4:** how configuration, path and returned history avoid aliasing.
- **5:** both test results, what JSON/SVG show and cannot establish, and the wheel filename after the next step.
- **6:** what Week 4 adds at the JSON boundary while preserving these invariants.

Save with **Ctrl+S**, then inspect the saved text:

Run - repository root

```
type artifacts\engineering_note.md
```

After recording the wheel filename in the next step, leave no placeholder or empty response. Put the final commit ID in LMS, not into the note: editing it afterward would require a different commit.

Procedure 3.8 Build and check the package

A wheel is an installable `.whl` file containing your completed package, unlike the editable link used while coding. Use the supplied build tools below. It still needs Python and the package's dependencies.

Check `dist` first. If it contains an old Week 3 wheel, remove only that old `.whl`, not your source. Build once more from the completed code:

Run - repository root

```
python -m build --wheel --no-isolation
dir /b dist\*.whl
```

Output from `dir /b`

```
ap_week03_planning-0.1.0-py3-none-any.whl
```

`--no-isolation` uses the active environment's build tools. `dir /b` lists filenames. Record the displayed filename in note section 5, save again, and run the supplied checker:

Run - repository root

```
python scripts/check_submission.py
```

The checker checks passing public/student tests, at least eight distinct student tests, filled note fields, regenerated JSON/SVG and one wheel. It imports the wheel in an isolated process using existing dependencies, not a fresh environment. Repair the item named by any FAIL. PASS does not confirm GitHub access, LMS submission or the quality of your explanations. The instructor reviews these and may test additional inputs and object relationships.

Procedure 3.9 Commit and push the completed work

Review your changes before selecting files for the commit:

Run - repository root

```
git status --short
git diff -- src/ap_week03_planning/domain.py
git diff -- src/ap_week03_planning/history.py
git diff -- tests/test_student_evidence.py
git add src/ap_week03_planning/domain.py
git add src/ap_week03_planning/history.py
git add tests/test_student_evidence.py artifacts dist
git diff --staged
```

Stage only your allowed edits and generated files. `git add` does not upload them. Press `q` to exit a diff pager, then commit and push:

Run - repository root

```
git commit -m "Complete Week 3 planning domain objects"
git push origin main
git rev-parse HEAD
git ls-remote origin refs/heads/main
git status --short
```

The final status must be empty and the full local and remote commit IDs must match. Check the actual SSUMechE access state for the LMS entry below.

7.6 Submit the Week 3 work through LMS

Use the same three fields as Weeks 1 and 2:

1. your private `applied-programming-w03-<student-id>` repository URL.
2. the full 40-character commit ID printed by `git rev-parse HEAD`.
3. confirmation of private visibility, actual SSUMechE access status and the revision to be graded.

Submit within one week of the Week 3 lab, following the exact LMS date/time. Do not wait for Week 4. Code, tests, note, generated files and wheel belong in the repository. No separate LMS ZIP or screenshot is required.

LMS entry - fictional example, replace all identity/hash values

```
Repository URL:
https://github.com/student-example/applied-programming-w03-20261234
Full commit ID:
0123456789abcdef0123456789abcdef01234567
Confirmation:
The repository is private. SSUMechE has active collaborator access.
The commit above is the revision to be graded.
```

Use your private URL and full pushed commit ID, not a template URL, local path, branch name, `latest` or short hash. Never include credentials. Later pushes do not replace the submitted revision.

Report a correct on-time invitation as pending if only instructor acceptance remains. This is not a student omission. Correct a missing or misaddressed invitation.

GitHub upload without LMS submission is not submission. Missing LMS submission, the required repository name or file structure, or mandatory submission items by the deadline results in zero. A failed numerical test is assessed for technical correctness, not automatically treated as a missing submission.